
Metal-Sci: A Scientific Compute Benchmark for Evolutionary LLM Kernel Search on Apple Silicon

Anonymous Authors¹

Abstract

We present METAL-SCI, a 10-task benchmark of scientific Apple Silicon Metal compute kernels spanning six optimization regimes (stencils, all-pairs, multi-field Boltzmann, neighbor-list molecular dynamics, multi-kernel PDE, FFT). Each task ships a CPU reference, a roofline-anchored fitness function, and a held-out generalization size. We pair the benchmark with a *lightweight harness for automatic kernel search* that runtime-compile each candidate, scores it against the roofline across multiple sizes, and feeds structured compile and per-size correctness diagnostics back to a frozen LLM driving a (1+1) evolutionary loop. We report matched single-model sweeps of Claude Opus 4.7, Gemini 3.1 Pro, and GPT-5.5 on M1 Pro: in-distribution self-speedups span $1.00\times$ to $10.7\times$. Beyond raw speedup, our central methodological claim is structural: the held-out gate $\Phi_{\mathcal{T}}$ — evaluated once at end-of-run on a configuration the agent never sees during search — functions as a cheap mechanical oversight primitive on this (1+1) coding agent, catching e.g. an Opus template `<uint D> HMC win` that returns wrong samples at unseen $d=24$ (Gemini’s runtime- d fallback and GPT-5.5’s $D \in \{8, 16, 24, 32\}$ enumeration generalize at $17.6\times$ and $18.6\times$), and a GPT-5.5 FFT3D best that wins in-distribution at $2.95\times$ but collapses to $0.23\times$ on a 256^3 held-out cube — a silent regression that the in-distribution score alone cannot see.

1. Introduction

LLM code-search systems such as FunSearch (Romera-Paredes et al., 2023), AlphaEvolve (Novikov et al., 2025),

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review at the Fifth Workshop on Deep Learning for Code (DL4C) at ICML 2026. Do not distribute.

and recent kernel-generation work (Ouyang et al., 2025) evaluate language models as *optimisers* of executable artefacts. The choice of artefact matters: KernelBench targets PyTorch ML kernels (GEMM, attention, convolution) on CUDA, where canonical implementations are heavily represented in pretraining data and the optimisation surface is dominated by tiling and warp-level reductions.

Scientific computing exposes a different surface. Stencils are bandwidth-bound and reward halo and temporal blocking. All-pairs interactions are compute-bound and reward register blocking with shared-memory cooperative loads. Lattice Boltzmann ping-pongs nine distribution functions per cell with non-trivial pull-stream indexing. Cell-list molecular dynamics touches irregular memory through atomic counters. Hamiltonian Monte Carlo runs many chains in parallel where register pressure scales with the problem dimension d . Grad-Shafranov solvers chain a max-reduction with a variable-coefficient stencil. 3D FFTs are dominated by data-shuffle/butterfly patterns and twiddle-factor caching rather than tiling. Each pattern stresses a distinct dimension of the compiler/memory hierarchy, and canonical optimizations (Datta et al., 2008; Nyland et al., 2007; Schönherr et al., 2011; Anderson et al., 2008; Govindaraju et al., 2008) differ regime-to-regime.

We target the Apple Silicon’s Metal compute pipeline. It is underrepresented in CUDA-centric training data, as frontier models reliably mishandle Metal-specific syntax such as the `[[max_total_threads_per_threadgroup]]` kernel attribute, a simple out-of-distribution generalization test. Its unified memory model removes host-device copy plumbing, enabling sub-second compile-run-verify cycles per candidate that are essential for fast evolutionary loops. And it supports rich simdgroup intrinsics (`simd_max`, `simd_broadcast`) and threadgroup-memory tiling, giving the LLM a non-trivial optimisation surface.

Contributions. (i) A 10-task scientific compute benchmark over six optimization regimes, each with a CPU reference, roofline-anchored fitness function, and multi-size generalization gate; (ii) a runtime-compiled harness that exposes compile errors, correctness diagnostics, and per-

size GPU timing back to the LLM; and (iii) three matched single-model sweeps (Claude Opus 4.7, Gemini 3.1 Pro, and GPT-5.5) on M1 Pro that operationalise the held-out gate $\Phi_{\mathcal{T}}$ as a cheap mechanical oversight primitive on the (1+1) coding agent: $\Phi_{\mathcal{T}}$ is never folded into any feed-back packet seen by the LLM during search and catches confidently-wrong outputs — silent correctness violations and silent regressions — that the in-distribution score $S_{\mathcal{T}}$ alone licenses.

Background and notation. *Apple GPU vocabulary.* A *kernel* is a .metal-source GPU function launched from the CPU; a *threadgroup* (TG) is Apple’s name for a CUDA thread block — cooperating threads sharing a scratchpad “threadgroup memory”; a *simdgroup* is the 32-thread SIMD lane group inside a threadgroup (Apple’s name for a CUDA warp); the *System-Level Cache* (SLC) is the CPU/GPU-shared last-level cache (~24 MB on M1 Pro) sitting between the on-chip caches and DRAM. *Roofline.* A kernel’s roofline (Williams et al., 2009) ceiling is the per-chip throughput upper bound implied by its arithmetic intensity (FLOPs per byte transferred). Kernels with low intensity are *bandwidth-bound* and cap at peak DRAM bandwidth (GB/s); high-intensity ones are *compute-bound* and cap at peak FP32 throughput (GFLOPS). We score candidates as a fraction of this hardware ceiling rather than against any hand-written baseline. *Evolution strategy.* ($\mu=1+\lambda=1$), also called (1+1), denotes the simplest evolution strategy: one parent, one mutated child per iteration; the child replaces the parent iff it scores strictly higher.

2. Benchmark tasks

METAL-SCI packages 10 tasks into six *optimisation regimes* (R1–R6, Table 1). The choice of regimes is the benchmark’s central design decision: each regime stresses a structurally distinct dimension of the GPU/memory hierarchy, and its canonical recipe (Datta et al., 2008; Nyland et al., 2007; Schönherr et al., 2011; Anderson et al., 2008; Govindaraju et al., 2008) does not transfer to its neighbours. A halo-blocking move that wins R1 is useless on R2 (register tiling), R4 (atomic contention), or R6 (intra-simdgroup butterflies). For an LLM, this is the structural reason recall alone cannot solve the suite: there is no template that wins everywhere, so the model has to actually *recognise the regime* from the seed and reach for the right lever.

Each task ships (a) a Metal seed kernel, (b) a CPU reference with task-specific tolerance, (c) three in-distribution training sizes plus one held-out size, and (d) a per-size roofline ceiling in GFLOPS (compute-bound) or GB/s (bandwidth-bound). Per-task equations, ceilings, and verification details are deferred to App. A; the prose below names the lever each regime tests.

R1 – Regular stencils. Bandwidth-bound updates over a structured grid: a 5-point heat-equation stencil (8 B/cell) and a 7-point leapfrog wave equation (12 B/cell). The lever is halo handling, marching-axis choice, and 2.5D temporal blocking. *wave3d* doubles as a NaN trap — a sign or indexing error compounds over many leapfrog steps, and 10 of Opus’s 13 correctness fails in Sec. 4 land here.

R2 – Compute-bound. $O(N^2)$ pair sums (nbody) and $O(d^2)$ matvec inside an L -step leapfrog (hmc), both running ~20 FLOPs per memory transaction so the ceiling is peak FP32 GFLOPS. The lever is register tiling and thread-group cooperative loads. *hmc* additionally probes the register-pressure boundary — per-thread $\mathbf{q}, \mathbf{p}, \mathbf{f}$ at $d=32$ is ~512 B — and verifies correctness statistically (sample mean and Frobenius covariance error vs. the target Gaussian).

R3 – Multi-field, exotic memory. *lbm* is a D2Q9 Lattice Boltzmann (pull-stream + BGK collision) with nine distribution fields per cell at 72 B/cell traffic; the lever is SoA layout, push/pull streaming choice, and algebraic factorisations of the BGK relaxation (App. C dissects an FMA fold Opus discovered). *ising* is 2D Ising checkerboard Metropolis MC at 2 B/site; a precomputed acceptance probability table and a counter-based Murmur-fmix32 PRNG yield bit-exact CPU/GPU agreement, so verification reduces to byte-equality on the spin array.

R4 – Irregular memory and atomics. Lennard-Jones MD with a cell-list spatial hash. Three kernels per step (*clear_cells* / *build_cells* / *step*); *build_cells* is an atomic scatter onto per-cell occupancy counters and the force kernel walks 27 neighbour cells with minimum-image periodic wrap. The lever is load balancing under uneven cell occupancy and atomic-contention mitigation.

R5 – Multi-kernel reductions. Picard iteration for the Grad-Shafranov fixed-boundary plasma equilibrium. Each outer step dispatches a max-reduction ($\psi_{\text{axis}} = \max \psi$) followed by a variable-coefficient 5-point stencil with a non-linear source. The lever is the choice of in-kernel reduction strategy — single-threadgroup vs. simdgroup-tree — and how cleanly the two kernels compose across the dispatch boundary.

R6 – Data-shuffle / butterfly. 3D complex-to-complex forward FFT, dispatched as three per-axis 1D FFTs with two ping-ponged buffers. Unlike R1/R2, the optimisation surface is *data movement* rather than arithmetic: bit-reversal vs. Stockham auto-sort, twiddle-factor caching, mixed-radix (radix-4, radix-8) butterflies for fewer barriers, and intra-simdgroup permutes via *simd_shuffle_xor*.

Table 1. The 10 METAL-SCI tasks. “Optimisation lever” names the dominant move an LLM must reach for in that regime; “Train sizes” / “Held-out” are passed to the harness verbatim. $N_x \times N_y$ grids written as N^2 when square; cube edges as N^3 . `saxpy` is a bandwidth smoke-test outside the regime structure, included to validate the harness.

Regime	Task	Optimisation lever	Train sizes	Held-out
R1 stencil	heat2d	halo, temporal blocking	$\{256, 512, 1024\}^2$	768^2
	wave3d	2.5D blocking, register pressure	$\{64, 160, 192\}^3$	128^3
R2 compute	nboddy	register tiling, threadgroup cooperative load	$N \in \{256, 1024, 2048\}$	512
	hmc	per-thread state vs. register file	$(d, K) \in \{(8, 16K), (16, 4K), (32, 1K)\}$	$(24, 2K)$
R3 multi-field	lbm	SoA layout, BGK algebraic fold	$\{64, 128, 256\}^2$	192^2
	ising	checkerboard MC, byte-exact verify	$\{256, 1024, 2048\}^2$	1536^2
R4 atomics	lj	cell-list scatter, atomic contention	$N \in \{1.7, 4.1, 10.6\}K$	2744
R5 multi-kernel	gradshaf	in-kernel reduction + var-coef stencil	$\{65, 257, 513\}^2$	129^2
R6 butterfly	fft3d	TG bank conflicts, mixed-radix, <code>simd_shuffle</code>	$\{32, 64, 128\}^3$	256^3
(smoke)	saxpy	DRAM saturation	$\{1, 16, 64\}M$	4M

The per-axis stride asymmetry (stride 1 along x vs. N and N^2 along y, z) makes threadgroup-memory bank-conflict avoidance a separate sub-lever.

3. Harness Design

Compile and dispatch. The harness uses PyObjC’s Metal bindings and runtime-compiles `.metal` source via `MTLDevice.newLibraryWithSource`, avoiding the offline `xcrun metal` toolchain; compile errors are returned as structured strings to the LLM. Buffers are allocated in unified memory; numpy views alias buffer contents with a single copy on read-back. All dispatches for a multi-size run share one `MTLCommandBuffer`, and timings come from `GPUEndTime - GPUStartTime` (3 warmup, 10 timed, median reported). The chip is detected from `sysctl` and looked up in a per-family table (M1 through M4) for peak FP32 GFLOPS and DRAM bandwidth; all runs here are on Apple M1 Pro (4500 GFLOPS, 200 GB/s).

Notation. A task $\mathcal{T} = (\sigma_{\mathcal{T}}, \Sigma_{\mathcal{T}}, \sigma_{\mathcal{T}}^*, c_{\mathcal{T}})$ packages a naive correct seed kernel $\sigma_{\mathcal{T}}$ (Metal source), three training sizes $\Sigma_{\mathcal{T}} = \{s_1, s_2, s_3\}$, one held-out size $\sigma_{\mathcal{T}}^* \notin \Sigma_{\mathcal{T}}$, and a per-size roofline $c_{\mathcal{T}}(s)$ in GFLOPS or GB/s. Evaluating a candidate κ at size s produces a correctness flag $\chi_{\mathcal{T}}(\kappa, s) \in \{0, 1\}$ (CPU reference within tolerance or not) and an achieved throughput $a_{\mathcal{T}}(\kappa, s)$; write $f_{\mathcal{T}}(\kappa, s) = a_{\mathcal{T}}(\kappa, s) / c_{\mathcal{T}}(s)$ for the per-size fraction-of-ceiling.

Scoring. The in-distribution score is the geometric mean of $f_{\mathcal{T}}$ over training sizes, gated on correctness on *every* size:

$$S_{\mathcal{T}}(\kappa) = \left(\prod_{s \in \Sigma_{\mathcal{T}}} f_{\mathcal{T}}(\kappa, s) \right)^{1/|\Sigma_{\mathcal{T}}|} \cdot \prod_{s \in \Sigma_{\mathcal{T}}} \chi_{\mathcal{T}}(\kappa, s). \quad (1)$$

The hard gate ($S_{\mathcal{T}}=0$ on any tolerance failure) prevents trading correctness for speed; the gmean across sizes dis-

courages overfit to one regime. The held-out gate, run only on the run’s incumbent at end-of-run, is the analogous quantity at the unseen size: $\Phi_{\mathcal{T}}(\kappa) = f_{\mathcal{T}}(\kappa, \sigma_{\mathcal{T}}^*) \cdot \chi_{\mathcal{T}}(\kappa, \sigma_{\mathcal{T}}^*)$. $S_{\mathcal{T}}$ is the agent’s optimisation target; $\Phi_{\mathcal{T}}$ is the external oversight signal it never sees.

Evolution loop. A frozen LLM $\mathcal{M}$ acts as a kernel synthesiser. Given a task-spec system prompt $p_{\mathcal{T}}$ and a feedback packet $\mathcal{F}_k$ summarising the previous candidate, the incumbent, and a short per-iteration history, $\mathcal{M}$ emits a Metal source

$$\kappa_{k+1} = \mathcal{M}(p_{\mathcal{T}}, q(\kappa_k, \kappa_k^*, \mathcal{F}_k)), \quad \kappa_0 = \sigma_{\mathcal{T}}, \quad (2)$$

which the harness compiles, dispatches across $\Sigma_{\mathcal{T}}$, and scores; the strict (1+1) rule promotes $\kappa_{k+1}^* = \kappa_{k+1}$ iff $S_{\mathcal{T}}(\kappa_{k+1}) > S_{\mathcal{T}}(\kappa_k^*)$, otherwise $\kappa_{k+1}^* = \kappa_k^*$. We formalise this compile-evaluate-promote (1+1) loop in Alg. 1 (App. B). Compile, pipeline, and per-size correctness errors are returned inside $\mathcal{F}_{k+1}$ as structured strings (the violating size, the error metric and value, and the compiler diagnostic if any). After K iterations the run terminates; both $S_{\mathcal{T}}(\kappa_K^*)$ and the held-out $\Phi_{\mathcal{T}}(\kappa_K^*)$ are reported. A single `call_llm` bridge dispatches Claude via the Agent SDK, Gemini via `google-genai`, and GPT-5.5 via the OpenAI Responses API at `reasoning_effort=high`.

4. Experiments

We run three matched single-model sweeps on Apple M1 Pro — `claude-opus-4-7` (*O*), `gemini-3.1-pro-preview` (*G*), and `gpt-5.5` (*G5*) — over the ten tasks at the same per-task iteration budget (10 each except `lbm` at 25 and `wave3d` at 15), $\mu=1+\lambda=1$, no human prompt intervention. Table 2 reports each model’s in-distribution self-speedup (best / seed, gmean over three training sizes) and a held-out evaluation in which the unmodified seed and each incumbent best are run on a single unseen problem size declared by the task spec (Sec. 2). The held-out columns are remea-

Table 2. Matched 10/15/25-iter sweeps on Apple M1 Pro: O = claude-opus-4-7, G = gemini-3.1-pro-preview, G5 = gpt-5.5. *In-dist.* $\times$ = best / seed, gmean over three training sizes; correctness is a hard gate. *Held-out frac.* = achieved/ceiling at the unseen size, measured in a single fresh session for all three models; *held-out* $\times$ = best / seed at that size. **Bold** marks regressions ($< 0.95\times$), the HMC correctness fail, and the GPT-5.5 FFT3D collapse.

Task	In-dist. $\times$			Held-out frac.			Held-out $\times$			Outcome
	O	G	G5	O	G	G5	O	G	G5	
saxpy	1.25	1.00	1.01	93%	78%	78%	1.17	0.98	0.98	saturated
heat2d	1.00	1.03	1.00	84%	98%	80%	0.86	1.01	0.82	saturated
wave3d	1.26	1.00	1.00	97%	87%	96%	1.00	0.90	0.99	saturated
ising	1.13	1.00	1.09	11%	12%	10%	0.94	0.99	0.88	flat
fft3d	1.03	1.19	2.95	42%	45%	8.5%	1.12	1.20	0.23	G5 silent regression
nbody	2.83	2.00	2.19	1.6%	2.0%	1.8%	1.24	1.50	1.37	generalises
gradshaf	1.89	2.89	1.93	5.4%	7.7%	5.0%	2.05	2.91	1.86	generalises
lj	1.77	1.98	1.62	0.31%	0.47%	0.34%	1.24	1.87	1.34	generalises
lbm	1.46	1.06	1.33	79%	93%	82%	0.97	1.16	1.01	tied at 192 ²
hmc	10.6	10.7	7.19	—	9.7%	10.2%	FAIL	17.6	18.6	O wrong at $d=24$; G,G5 clean

sured in a single fresh session for all three models so the absolute fraction-of-ceiling numbers stay apples-to-apples; we observed that single-size held-out fractions can shift by tens of percentage points across sessions due to GPU thermal state and SLC residency, so we anchor on the within-session ratios.

In-distribution. Self-speedups span $1.00\times$ to $10.7\times$. The HMC step (Fig. 1) is the high end for *Opus* and *Gemini*: each independently introduces a `template <uint D> worker dispatched on runtime d` that takes $d=8$ from ~ 120 to ~ 970 GFLOPS by enabling full unroll of the inner Aq matvec, and the two top scores agree to within 1.4% (O 0.0932 vs G 0.0870); GPT-5.5 reaches the same regime more cautiously at $7.2\times$ (G5 0.0634). Outside HMC the models split: *Opus* wins on saxpy (1.25), nbody (2.83), lbm (1.46), and wave3d (1.26); *Gemini* wins on gradshaf (2.89) and lj (1.98); **GPT-5.5 wins fft3d outright at $2.95\times$** (vs 1.19 *Gemini*, 1.03 *Opus*), the largest in-distribution gap any single model opens on the suite, and is competitive on nbody (2.19), gradshaf (1.93), ising (1.09), and lbm (1.33). The *Opus*–*Gemini* split correlates with the type of optimisation lever: “tune the same algorithm tighter” (BW saturation, BGK fold, leapfrog ILP) favours *Opus*, while “find a different algorithm” (simdgroup-tree reduction, twiddle caching, neighbour-list reorganisation) favours *Gemini* (see App. C for code-level diffs on lbm and fft3d, plus GPT-5.5’s FFT3D direct-DFT fallback and HMC defensive enumeration). GPT-5.5 sits closer to *Gemini* in temperament — it explores aggressive restructurings, which on fft3d pays off in-distribution but, as the held-out column makes visible, at the cost of a sharp overfit (§ below). Saturated tasks (saxpy, heat2d, wave3d) sit above 78% of effective DRAM ceiling on the seed; saxpy and heat2d primarily validate the harness, leaving the search loop little to do — and heat2d shows the score is a *hard* signal: on both *Opus* and GPT-5.5 zero candidates strictly dominated the seed across all

three sizes and the incumbent stayed at iter 0. Across all three sweeps, *Gemini* had *zero* correctness failures across its 95 fresh-run candidates; *Opus* had 13 across 110 candidates (notably wave3d 10/15: multi-step leapfrog amplifies any sign or indexing error into NaN); GPT-5.5 had 2 across 120 candidates, the second-lowest correctness-failure rate of the three after *Gemini*.

Held-out generalisation is sharper than in-distribution — and the three models overfit asymmetrically. Three populations separate. (i) *Generalises*: nbody, gradshaf, lj, and (for O and G) fft3d. Gradshaf is the standout where all three models extrapolate cleanly ($2.05\times$ O, $2.91\times$ G, $1.86\times$ G5); on lj only *Gemini* exceeds its in-distribution speedup ($1.87\times$ at $N=2744$); *Opus* and GPT-5.5 hold partial gains ($1.24\times$ and $1.34\times$ respectively). (ii) *Saturated*: saxpy, heat2d, wave3d, ising, and lbm 192². lbm is the cleanest reclassification under our refreshed session: where an earlier measurement at the held-out grid showed both *Opus* and *Gemini* regressing to $0.41\times/0.50\times$, the single-session three-way remeasurement above puts all three within the $0.97\times$ – $1.16\times$ band, with *Gemini* even gaining a modest $1.16\times$. The 192-square sits between the 128² and 256² training sizes, on a multiple of 32, and SLC residency at small grids dominates the absolute fraction-of-ceiling — enough to flip the within-task ordering session-to-session. We treat lbm as held-out-saturated rather than as a regression exemplar. (iii) *Overfits*, with the most consequential disagreements between the three models. **HMC** is the sharpest correctness case: *Opus*’s template specialisation dispatches `if (d==8) run<8>() ... else run<32>()`, so $d=24$ lands in the $D=32$ branch, per-thread `q[32], p[32]` and the unrolled matvec process 32 entries against 24-entry data, and sample covariance lands $\sim 10\sigma$ off target. *Gemini* pairs the template-D speedup with a runtime- d leapfrog fallback; GPT-5.5 goes one step further and enumerates $D \in \{8, 16, 24, 32\}$ ex-

licitly (App. C), covering the held-out dimension with its own fully-unrolled template instance *and* a runtime- d safety net for any other d . Both generalise to $d=24$ at $\sim 10\%$ of FP32 peak ($17.6\times$ G, $18.6\times$ G5). **FFT3D is the sharpest performance case for the new model:** GPT-5.5’s iter-10 best wins the in-distribution gmean at $2.95\times$ but on the held-out 256^3 cube — past the largest training size 128^3 — it collapses to $0.23\times$ of seed (8.5% of effective ceiling vs Opus’s 42% and Gemini’s 45% on the same configuration). The kernel relies on a fixed-twiddle, fixed-geometry layout tuned for $N\leq 128$; at $N=256$ the register pressure and tg-memory budget no longer fit, and the fallback path is dramatically slower than the seed’s textbook Stockham. This is the cleanest silent-regression instance in the sweep: $S_{\mathcal{T}}$ alone licenses a $2.95\times$ win, $\Phi_{\mathcal{T}}$ surfaces a $0.23\times$ deployment-grade slowdown. **GPT-5.5** is never strictly worse than Opus on held-out correctness (both clean except Opus’s HMC fail) and on generalisation falls between Opus and Gemini on most tasks — but its FFT3D collapse is the largest single held-out swing in the table and exemplifies the oversight value of $\Phi_{\mathcal{T}}$ on a non-Anthropropic, non-Google frontier model.

Recurring Metal-grammar failures. Compile-error patterns are similar across the three models. `[[max_total_threads_per_threadgroup(N)]]` is mis-placed (after the parameter list, or as a standalone statement, instead of on the kernel void declaration) on Opus across five tasks — reproduced from earlier opus-4-6 runs — and GPT-5.5 hits the same attribute placement error on its first saxpy iteration. `half` is reserved as MSL’s fp16 type, breaking `uint half = N >> 1u;`; C++ lambdas are unsupported. The three sweeps differ in volume rather than kind: 12/110 compile fails for Opus, 22/95 for Gemini, 12/120 for GPT-5.5 — GPT-5.5 has the lowest compile-fail rate (10%) and the second-lowest correctness-fail rate (2/120) of the three; Gemini keeps its perfect 0/95 correctness record. Wall time scales the opposite way: Opus 0.6 min/iter, Gemini 3.5 min/iter, GPT-5.5 6.6 min/iter (`reasoning_effort=high`); GPT-5.5’s wider exploration and longer reasoning context costs roughly $2\times$ Gemini and $11\times$ Opus per iteration at matched budget.

5. Discussion

The held-out gate as agent oversight. The benchmark’s (1+1) loop is, modulo terminology, an autonomous coding agent: it reads a Metal source, edits it, runs it, and self-promotes its own outputs based on an internal fitness signal. A human merging the agent’s incumbent into a downstream codebase sees only the in-distribution score $S_{\mathcal{T}}$ the agent reports — and $S_{\mathcal{T}}$ is gameable in two ways the held-out gate $\Phi_{\mathcal{T}}$ (Sec. 3) catches. **Silent correctness**

violation. On HMC, Opus’s incumbent hits $10.6\times$ with all in-distribution checks green; held out at $d=24$ — a problem dimension that sits between two training values and is not flagged as out-of-scope — the same code returns samples whose covariance is off by $\sim 10\sigma$. A user who trusted the reported number would ship a sampler that looks calibrated and isn’t. **Silent regression.** On FFT3D GPT-5.5 reports an in-distribution win of $2.95\times$ — the largest single-model in-distribution gap in our sweep — that flips to a $0.23\times$ slowdown at the held-out 256^3 cube (the one configuration past the largest training size 128^3). The agent confidently labels its iter-10 output as a $\sim 3\times$ “improvement” over the seed; the held-out gate shows it is $4\times$ slower in deployment. Both failures share a structural shape: the agent specialised against the visible workload and reported a number that does not carry over. A single auxiliary problem instance per task — one extra dispatch, seconds of GPU time — is enough to surface both. This reframes the contribution toward the workshop’s verifiability/oversight agenda: METAL-SCI’s value lies less in providing a hard benchmark and more in instantiating a cheap, mechanical trust contract on top of an autonomous coding agent. The roofline anchor, per-size tolerance, and geometric mean inside $S_{\mathcal{T}}$ are the agent’s scoring machinery; $\Phi_{\mathcal{T}}$ — evaluated once on $\sigma_{\mathcal{T}}^*$ at end-of-run, never folded into any feedback packet $\mathcal{F}_k$ — is the lightweight oversight primitive that lets a human (or a downstream automated reviewer) catch confidently-wrong agent code before deployment.

Other observations. A roofline-anchored score answers “how close are we to the hardware?” in physical units; the gmean across sizes is hard to game by overfitting one regime. The three-model asymmetry is concrete: at matched iteration budgets the in-distribution scores are close, but each model fails the held-out gate in a different shape. Opus loses correctness at HMC $d=24$, GPT-5.5 loses performance at FFT3D 256^3 , and Gemini — the only model with zero correctness failures across the entire 95-candidate budget and no held-out collapse beyond a borderline $0.90\times$ on wave3d — is the most robust of the three. Wall time orders the other way: Opus is $\sim 11\times$ faster per iteration than GPT-5.5 and $\sim 6\times$ faster than Gemini at matched budgets, which makes Opus the cheapest run when in-distribution speedup is the only deliverable, and Gemini/GPT-5.5 the safer choice when the output ships across configurations the training set did not cover. Apple Silicon’s unified memory halves the host-side scaffolding compared to CUDA, enabling sub-second compile-run-verify cycles per candidate — which matters more than it sounds: an evolutionary loop with tens of candidates needs a fast *harness*, not a fast kernel. Metal’s smaller, quirrier surface also surfaces OOD failures of CUDA-trained LLMs on tasks that are otherwise textbook.

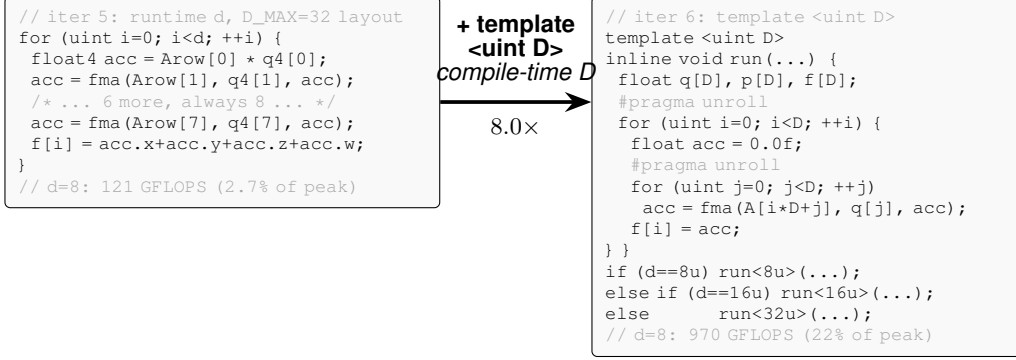


Figure 1. Paradigmatic candidate evolution: HMC, Opus 4.7, iter 5 $\rightarrow$ 6. Both Opus and Gemini independently arrive at this structural change. The lever is one declaration — `template <uint D>` with runtime-dispatched instantiations on d . Iter 5 had a manually-unrolled float4 inner loop against a fixed $D_{\max}=32$ layout (over-computing at $d=8$, plus a 4-way horizontal sum); iter 6 sizes q, p, f exactly to D so both loops fully unroll into scalar FMAs — moving $d=8$ from 121 to 970 GFLOPS in one iteration after five had stalled at 2–4% of peak. Opus enumerated $D \in \{8, 16, 32\}$ and broke correctness at the held-out $d=24$; Gemini kept a runtime- d fallback and generalised cleanly.

Limitations and future work. The static per-chip ceilings do not account for SLC residency at small sizes (see the LBM 256² result above); a workload-aware roofline (Williams et al., 2009) per task and per size would tighten the score. The single-population (1+1) loop plateaus within 10–25 iterations on most tasks — an island-model or FunSearch-style (Romera-Paredes et al., 2023) archive with a novelty signal (Novikov et al., 2025) is the natural next step, particularly for the irregular-memory tasks. Future tasks include sparse linear algebra (SpMV, CG) and cross-chip generalisation (evolve on M2 Pro, evaluate on M3 Max).

References

- Anderson, J. A., Lorenz, C. D., and Travesset, A. General purpose molecular dynamics simulations fully implemented on graphics processing units. *Journal of computational physics*, 227(10):5342–5359, 2008.
- Datta, K., Murphy, M., Volkov, V., Williams, S., Carter, J., Oliker, L., Patterson, D., Shalf, J., and Yelick, K. Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. In *Proceedings of the 2008 ACM/IEEE Conference on Supercomputing, SC '08*. IEEE Press, 2008. ISBN 9781424428359.
- Govindaraju, N. K., Lloyd, B., Dotsenko, Y., Smith, B., and Manferdelli, J. High performance discrete fourier transforms on graphics processors. *SC*, 8:1–12, 2008.
- Guo, P., Zhu, C., Chen, S., Liu, F., Lin, X., Lu, Z., and Zhang, Q. Evoengineer: Mastering automated cuda kernel code evolution with large language models. *arXiv preprint arXiv:2510.03760*, 2025.
- Kalade, S. and Schelle, G. NPUEval: Optimizing npu kernels with llms and open source compilers. *arXiv preprint arXiv:2507.14403*, 2025.
- Lange, R. T., Sun, Q., Prasad, A., Faldor, M., Tang, Y., and Ha, D. Towards robust agentic cuda kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*, 2025.
- Li, J., Li, S., Gao, Z., Shi, Q., Li, Y., Wang, Z., Huang, J., WangHaojie, W., Wang, J., Han, X., et al. Triton-bench: Benchmarking large language model capabilities for generating triton operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 23053–23066, 2025.
- Nie, J., Wu, H., Lai, Y., Cao, Z., Zhang, C., Lou, B., Wang, E., Cheng, J., Jones, T. M., Mullins, R., et al. Kernelcraft: Benchmarking for agentic close-to-metal kernel generation on emerging hardware. *arXiv preprint arXiv:2603.08721*, 2026.
- Novikov, A., Vū, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Nyland, L., Harris, M., and Prins, J. Fast n-body simulation with cuda. *gpu gems 3*, 2007.
- Ouyang, A., Guo, S., Arora, S., Zhang, A. L., Hu, W., Re, C., and Mirhoseini, A. Kernelbench: Can LLMs write efficient GPU kernels? In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=yeoNliQTlx>.
- Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J., Wang, P., Fawzi, O., Kohli, P., and Fawzi,

A. Mathematical discoveries from program search with large language models. *Nature*, 2023. doi: 10.1038/s41586-023-06924-6.

Saroufim, M., Wang, J., Maher, B., Paliskara, S., Wang, L., Sefati, S., and Candales, M. Backendbench: An evaluation suite for testing how well llms and humans can write pytorch backends, 2025. URL <https://github.com/meta-pytorch/BackendBench>.

Schönherr, M., Kucher, K., Geier, M., Stiebler, M., Freudiger, S., and Krafczyk, M. Multi-thread implementations of the lattice boltzmann method on non-uniform grids for cpus and gpus. *Computers & Mathematics with Applications*, 61(12):3730–3743, 2011.

Wen, Z., Zhang, Y., Li, Z., Liu, Z., Xie, L., and Zhang, T. Multikernelbench: A multi-platform benchmark for kernel generation. *arXiv e-prints*, pp. arXiv-2507, 2025.

Williams, S., Waterman, A., and Patterson, D. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

A. Task formulations

This appendix gives the per-task equations, ceilings, and tolerances summarised in Section 2. Tasks are grouped by the regimes R1–R6 of Table 1; training and held-out sizes match the table.

A.1. R1: Regular stencils

heat2d. Two-dimensional heat equation, 5-point stencil on $(N_x \times N_y)$ grid with Dirichlet boundaries. Per timestep, per interior cell:

$$u_{i,j}^{n+1} = u_{i,j}^n + \alpha(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n). \quad (3)$$

Bandwidth-bound at 8 B/cell.

wave3d. Three-dimensional acoustic wave equation, 7-point isotropic Laplacian, leapfrog in time:

$$u_{i,j,k}^{n+1} = 2u_{i,j,k}^n - u_{i,j,k}^{n-1} + \alpha(u_{i\pm 1,j,k}^n + u_{i,j\pm 1,k}^n + u_{i,j,k\pm 1}^n - 6u_{i,j,k}^n), \quad (4)$$

where each $\pm$ expands to two terms. CFL stability requires $\alpha = (c\Delta t/\Delta x)^2 < 1/3$ (we use 0.18). 12 B/cell unique DRAM traffic.

A.2. R2: Compute-bound

nbody. All-pairs gravitational N -body with leapfrog integration. Per body i per step:

$$\mathbf{a}_i = G \sum_j m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \varepsilon^2)^{3/2}}, \quad \mathbf{v} += \mathbf{a}\Delta t, \quad \mathbf{r} += \mathbf{v}\Delta t. \quad (5)$$

Self-interaction is masked by softening ε . ~ 20 FLOPs per pair; ceiling at peak FP32 GFLOPS.

hmc. Hamiltonian Monte Carlo on an anisotropic Gaussian target, $U(q) = \frac{1}{2}q^\top Aq$ with $A = \Sigma^{-1}$. One thread per chain; many chains in parallel. Each step draws $p \sim \mathcal{N}(0, I)$ and runs L leapfrog inner steps with stepsize ε ,

$$p \leftarrow p - \frac{\varepsilon}{2}Aq, \quad q \leftarrow q + \varepsilon p, \quad p \leftarrow p - \varepsilon Aq, \dots, \quad p \leftarrow p - \frac{\varepsilon}{2}Aq, \quad (6)$$

followed by a Metropolis accept/reject with $\log u_{\text{acc}} < -\Delta H$. Correctness is verified statistically (sample mean and Frobenius covariance error vs. the target). The three training sizes $(d, K) \in \{(8, 16K), (16, 4K), (32, 1K)\}$ probe the register-pressure boundary; at $d=32$ per-thread state (~ 512 B) competes with the register file.

A.3. R3: Multi-field, exotic memory

lbm. D2Q9 Lattice Boltzmann, fused pull-stream + BGK collision, periodic BC. With velocity table $\mathbf{c}_k$ and weights w_k , per cell per step:

$$f_k^{\text{str}}(\mathbf{x}) = f_k^{\text{in}}(\mathbf{x} - \mathbf{c}_k), \quad \rho = \sum_k f_k^{\text{str}}, \quad \mathbf{u} = \frac{1}{\rho} \sum_k \mathbf{c}_k f_k^{\text{str}}, \quad (7)$$

$$f_k^{\text{eq}} = w_k \rho \left(1 + 3(\mathbf{c}_k \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{c}_k \cdot \mathbf{u})^2 - \frac{3}{2}\|\mathbf{u}\|^2 \right), \quad (8)$$

$$f_k^{\text{out}} = f_k^{\text{str}} - \tau^{-1}(f_k^{\text{str}} - f_k^{\text{eq}}). \quad (9)$$

Storage is SoA: $f[k N_x N_y + j N_x + i]$. 72 B/cell DRAM traffic.

ising. Two-dimensional Ising model, checkerboard Metropolis Monte Carlo on a periodic $N_x \times N_y$ lattice with $\sigma \in \{\pm 1\}$ stored as int8. Per attempt at site (i, j) :

$$h = \sigma_{i\pm 1,j} + \sigma_{i,j\pm 1} \in \{-4, \dots, 4\}, \quad \Delta E = 2J\sigma h, \quad \text{accept} \iff u < \exp(-\beta \Delta E). \quad (10)$$

A precomputed five-entry p_{accept} table and a counter-based Murmur-fmix32 PRNG yield bit-exact CPU/GPU agreement; verification is byte-equality on the spin array. 2 B/site/sweep ceiling.

A.4. R4: Irregular memory and atomics

lj. Lennard-Jones molecular dynamics with a cell-list spatial hash. Three kernels per step — `clear_cells`, `build_cells` (atomic scatter), `step` (27-neighbour-cell pair force + integration):

$$\mathbf{F}_i = \sum_{j \in \mathcal{N}(i)} -24(2r_{ij}^{-12} - r_{ij}^{-6}) r_{ij}^{-2} (\mathbf{r}_j - \mathbf{r}_i), \quad r_{ij} < r_{\text{cut}} = 2.5, \quad (11)$$

with minimum-image periodic wrap. Cell occupancy is built via `atomic_fetch_add` on a per-cell counter.

A.5. R5: Multi-kernel reductions

gradshaf. Grad-Shafranov fixed-boundary equilibrium via Picard iteration, two kernels per outer step: an interior max-reduction $\psi_{\text{axis}} = \max_{(i,j) \in \text{int}} \psi$, then a variable-coefficient 5-point stencil with nonlinear source:

$$\psi_{\text{norm}} = \psi / \psi_{\text{axis}}, \quad J = R p_{\text{axis}} \cdot 4\psi_{\text{norm}}(1 - \psi_{\text{norm}}) \cdot \mathbf{1}[0 < \psi_{\text{norm}} < 1], \quad (12)$$

$$\Delta^* \psi = a_W \psi_W + a_E \psi_E + a_N \psi_N + a_S \psi_S + a_C \psi_C, \quad (13)$$

$$\psi^{n+1} = \psi^n + \omega(-\mu_0 R J - \Delta^* \psi) / a_C, \quad (14)$$

with R -dependent $a_{W,E} = 1/dR^2 \pm 1/(2RdR)$, $a_{N,S} = 1/dZ^2$, $a_C = -2(1/dR^2 + 1/dZ^2)$.

A.6. R6: Data-shuffle / butterfly

fft3d. 3D complex-to-complex forward FFT, fp32, on a power-of-two cube of side N . Convention is unnormalised (matches `numpy.fft.fftn`); storage is row-major `float2[N][N][N]`. Three named kernels — `fft3d_x`, `fft3d_y`, `fft3d_z` — are dispatched in sequence with two ping-ponged buffers, each performing a length- N 1D FFT per thread-group of N cooperating threads:

$$Y[k_1, k_2, k_3] = \sum_{n_1, n_2, n_3=0}^{N-1} X[n_1, n_2, n_3] \exp\left(-\frac{2\pi i}{N}(k_1 n_1 + k_2 n_2 + k_3 n_3)\right). \quad (15)$$

Sizes $N \in \{32, 64, 128\}$ cover three working-set regimes: 32^3 (~ 256 KB) is SLC-resident and compute-bound; 128^3 (~ 16 MB) DRAM-binds. $\sim 5N \log_2 N$ FLOPs per 1D FFT; effective DRAM traffic 96 B/cell across the three axis passes (16 B read + 16 B write per pass). Verification is max-norm against `numpy.fft.fftn` on a fixed seeded Gaussian input, tolerance $10^{-3} + 10^{-3} \|Y\|_{\infty}$.

B. Evolution loop pseudocode

Algorithm 1 formalises the harness of Sec. 3. The EVALUATE subroutine is the compile–run–score pipeline: it returns either a structured failure (compile or per-size correctness, with the violating size s and the error metric so $\mathcal{M}$ can correct course inside $\mathcal{F}_{k+1}$) or a successful score $S_{\mathcal{T}}(\kappa)$ together with per-size fractions-of-ceiling. The held-out $\Phi_{\mathcal{T}}(\kappa_K^*)$ is computed once at end-of-run on the unseen size $\sigma_{\mathcal{T}}^*$ — never returned in any $\mathcal{F}_k$ — and is the oversight signal of Sec. 4.

C. Code-level evidence for the in-distribution split

The comparative claim in Section 4 is mechanistic — Opus wins by tightening the same algorithm, Gemini wins by reaching for a different one. Figure 1 instantiates that claim on HMC. This appendix instantiates it on two more tasks — one Opus-favoured (lbm), one Gemini-favoured (fft3d) — with the actual diff between each model’s incumbent best.

C.1. LBM: Opus tightens BGK with FMA folds and a pinned threadgroup

Both models share the seed’s pull-stream + BGK structure. The diff is local to the collision step and to the kernel attribute (Fig. 2). Opus precomputes $A = \text{fma}(-1.5, \|\mathbf{u}\|^2, 1)$ once per cell, factors the per-direction equilibrium as $A + c \cdot u(3 + 4.5c \cdot u)$ — two FMAs — and folds the relaxation into a third $\text{fma}(1 - \omega, f_k, \omega W_k \rho t)$, yielding nine manually unrolled blocks. It also pins `[max_total_threads_per_threadgroup(64)]` on the kernel declaration, picking a 32×2 tile aligned to the simdgroup width. Gemini keeps the textbook BGK formula $w_k \rho(1 + 3c \cdot u + 4.5(c \cdot u)^2 - 1.5\|\mathbf{u}\|^2)$ inside

Algorithm 1 METAL-SCI evolution loop ($\mu=1+\lambda=1$).

Require: task $\mathcal{T}=(\sigma_{\mathcal{T}}, \Sigma_{\mathcal{T}}, \sigma_{\mathcal{T}}^*, c_{\mathcal{T}})$, LLM $\mathcal{M}$, system prompt $p_{\mathcal{T}}$, iterations K

Ensure: incumbent κ_K^* , in-dist score $S_{\mathcal{T}}(\kappa_K^*)$, held-out $\Phi_{\mathcal{T}}(\kappa_K^*)$

```

1:  $\kappa_0^* \leftarrow \sigma_{\mathcal{T}}; \mathcal{F}_0 \leftarrow \text{EVALUATE}(\sigma_{\mathcal{T}}, \mathcal{T})$  ▷ seed is initial incumbent
2: for  $k = 1, \dots, K$  do
3:    $\kappa_k \leftarrow \mathcal{M}(p_{\mathcal{T}}, q(\kappa_{k-1}, \kappa_{k-1}^*, \mathcal{F}_{k-1}))$  ▷ propose
4:    $r_k \leftarrow \text{EVALUATE}(\kappa_k, \mathcal{T})$ 
5:    $\mathcal{F}_k \leftarrow (\kappa_k, r_k)$  ▷ structured feedback for next iter
6:   if  $r_k.\text{score defined} \wedge r_k.\text{score} > S_{\mathcal{T}}(\kappa_{k-1}^*)$  then
7:      $\kappa_k^* \leftarrow \kappa_k$  ▷ (1+1) promote
8:   else
9:      $\kappa_k^* \leftarrow \kappa_{k-1}^*$ 
10:  end if
11: end for
12:  $\Phi_{\mathcal{T}}(\kappa_K^*) \leftarrow f_{\mathcal{T}}(\kappa_K^*, \sigma_{\mathcal{T}}^*) \cdot \chi_{\mathcal{T}}(\kappa_K^*, \sigma_{\mathcal{T}}^*)$  ▷ never given to  $\mathcal{M}$ 
13: return  $\kappa_K^*, S_{\mathcal{T}}(\kappa_K^*), \Phi_{\mathcal{T}}(\kappa_K^*)$ 

14: function  $\text{EVALUATE}(\kappa, \mathcal{T})$ 
15:    $\text{lib}, e_c \leftarrow \text{COMPILE}(\kappa)$ 
16:   if  $e_c \neq \emptyset$  then return (compile_fail,  $e_c$ )
17:   end if
18:   for  $s \in \Sigma_{\mathcal{T}}$  do
19:      $(\chi_s, a_s) \leftarrow \text{RUN}(\text{lib}, s)$ 
20:     if  $\chi_s = 0$  then return (correct_fail,  $s$ , error metric)
21:   end if
22:   end for
23:    $S \leftarrow (\prod_{s \in \Sigma_{\mathcal{T}}} a_s / c_{\mathcal{T}}(s))^{1/|\Sigma_{\mathcal{T}}|}$ 
24:   return (score:  $S$ ,  $\{a_s / c_{\mathcal{T}}(s)\}_{s \in \Sigma_{\mathcal{T}}}$ )
25: end function

```

a #pragma unroll for (k=0...8), no FMA folds, no A-extraction, no threadgroup pin. The two are competitive at 64^2 (O 0.34, G 0.32) and Gemini actually wins at 128^2 (O 0.47, G 0.51); Opus pulls ahead at 256^2 (O 1.22, G 1.03), the cache-resident regime where the pinned 32×2 geometry and the FMA-folded BGK extract every issued instruction — and that is the size with the largest absolute fraction-of-ceiling, so it dominates the gmean.

C.2. FFT3D: Gemini swaps the algorithm to `simd_shuffle_xor`

The fft3d gmean gap is larger (0.282 vs 0.167, $1.7\times$), and the diff is not a tighter version of the same kernel — Fig. 3 shows two different algorithms. Opus implements a textbook Stockham auto-sort radix-4 FFT: every stage ping-pongs through threadgroup memory, with a barrier between stages. Gemini observes that for the first five Cooley–Tukey stages the butterfly partner of lane i is at lane $i \oplus 2^{s-1}$ where $s \in \{1, \dots, 5\}$, and $2^{s-1} < 32$ — so it can be fetched with `simd_shuffle_xor`, no shared memory and no barrier. Only stages $s \geq 6$ fall back to threadgroup memory. The Apple GPU’s `simdgroup` width is exactly 32: this trick is Metal-specific (`simd_shuffle_xor` maps to a single hardware permute), and worth ~ 5 barriers per length- N FFT, ~ 15 per 3D transform. The same “find a different memory primitive” move shows up smaller-scale on gradshaf: both models converge on a `simdgroup`-tree max-reduction, but Gemini additionally casts `psi` to `float4*` and reads four scalars per transaction in the inner sweep, halving the number of issued loads on the dominant kernel.

C.3. GPT-5.5 FFT3D: hand-coded fast paths for $N \leq 128$, $O(N^2)$ direct DFT for everything else

GPT-5.5’s iter-10 FFT3D best wins the in-distribution gmean ($2.95\times$, vs Opus $1.03\times$ and Gemini $1.19\times$) with hand-coded `fft_line_32/64/128` routines that use `simd_shuffle_xor` (the same `simdgroup` primitive Gemini reaches for) for the intra-`simdgroup` stages and a precomputed constant `float2 w128[64]` twiddle table for the per-stage

```

550 // Opus: BGK fold + pinned geometry
551 [[max_total_threads_per_threadgroup(64)]]
552 kernel void lbm_step(...) {
553     // pull-stream f0..f8, moments rho, ux, uy
554     ...
555     float A = fma(-1.5f, usq, 1.0f);
556     float orWS = (omega * W_S) * rho;
557     // k=5: cu = ux+uy
558     {
559         float cu = ux + uy;
560         float t = A + cu * fma(4.5f, cu, 3.0f);
561         q5[idx] = fma(one_m_w, f5, orWS * t);
562     }
563     // 8 more directions, same shape ...
564 }
    
```

```

550 // Gemini: textbook BGK in unrolled loop
551 kernel void lbm_step(...) {
552     // pull-stream f[k], moments rho, ux, uy ...
553     float usq = ux*ux + uy*uy;
554     float inv_tau = 1.0f / tau;
555     #pragma unroll
556     for (int k = 0; k < 9; ++k) {
557         float cu = CX[k]*ux + CY[k]*uy;
558         float feq = W[k] * rho *
559             (1.0f + 3.0f*cu + 4.5f*cu*cu
560              - 1.5f*usq);
561         f_out[k*N+idx] =
562             f[k] - inv_tau*(f[k]-feq);
563     }
564 }
    
```

Figure 2. LBM iter-23 best, Opus (left) vs Gemini iter-13 best (right). Opus extracts A once, folds f_k^{eq} into two FMAs per direction and the relaxation into a third, and pins the threadgroup geometry; Gemini stays with the canonical BGK formula and the default geometry. In-distribution gmean: Opus 0.576 vs Gemini 0.553.

```

565 // Opus: Stockham radix-4, all in tg memory
566 for (uint s = 0u; s < stages4; ++s) {
567     // load 4 inputs from cur[]
568     float2 x0 = cur[b + 0u*Nq];
569     float2 x1 = cur[b + 1u*Nq];
570     float2 x2 = cur[b + 2u*Nq];
571     float2 x3 = cur[b + 3u*Nq];
572     // twiddle multiplies (skip s=0)
573     if (s != 0u) { ... }
574     // 4-point DFT, write to nxt[]
575     threadgroup_barrier(mem_threadgroup);
576     swap(cur, nxt);
577 }
    
```

```

565 // Gemini: simd_shuffle_xor for stages 1..5
566 // (no shared mem, no barrier)
567 if (logN >= 1u) {
568     float2 v = simd_shuffle_xor(u, 1u);
569     u = ((i & 1u) == 0u) ? (u+v) : (v-u);
570 }
571 // stages 2..4 analogous (xor 2, 4, 8)
572 if (logN >= 5u) {
573     float2 v = simd_shuffle_xor(u, 16u);
574     // twiddle, butterfly ...
575     u = ((i & 16u) == 0u) ? (u+t) : (v-t);
576 }
577 // stages s>=6: fall back to tg memory
578 for (uint s = 6u; s <= logN; ++s) { ... }
    
```

Figure 3. FFT3D iter-10 best, Opus (left) vs Gemini iter-10 best (right). Opus implements Stockham radix-4 with threadgroup-memory ping-pong and a barrier per stage; Gemini exploits the 32-wide simdgroup to do the first five stages with `simd_shuffle_xor`, eliminating five barriers per 1D FFT. In-distribution gmean: Opus 0.167 vs Gemini 0.282.

multiplies. The held-out collapse comes from a single line in the dispatch: when N does not match one of $\{32, 64, 128\}$ the kernel falls into a textbook direct $O(N^2)$ DFT (Fig. 4). At $N=256$ that path performs 256 complex multiplies per output element instead of the seed’s $\log_2(256)=8$ butterfly stages; per 1D line this is a $\sim 32\times$ arithmetic blow-up, and three 1D passes per 3D transform compound it. The held-out gate $\Phi_{\mathcal{T}}$ surfaces the resulting $0.23\times$ slowdown that the in-distribution score $S_{\mathcal{T}}$ alone licenses.

C.4. GPT-5.5 HMC: defensive D enumeration covering $d=24$

On HMC GPT-5.5 lands at a structurally different point than either Opus or Gemini. Opus enumerates only $D \in \{8, 16, 32\}$ and dispatches $d=24$ to the $D=32$ branch (Fig. 1), silently running an unrolled matvec against the wrong size. Gemini keeps a pure runtime- d leapfrog with no template specialisation, paying for safety with in-distribution throughput. GPT-5.5 takes the union: an explicit $D \in \{8, 16, 24, 32\}$ enumeration with fully-templated instances for each, plus a generic runtime- d fallback for anything outside that set (Fig. 5). The $D=24$ branch is the cleanest defensive move in the suite — the held-out dimension is treated as a first-class instantiation rather than a special case to round up or fall back on.

D. Related work

D.1. LLM-driven kernel-generation benchmarks

A wave of benchmarks has emerged for evaluating LLMs as generators of high-performance GPU kernels. *KernelBench* (Ouyang et al., 2025) targets PyTorch ML kernels (GEMM, attention, convolution, normalisation, activation) on CUDA and scores single-shot generation by speedup against the PyTorch eager baseline. *TritonBench* (Li et al., 2025) extends to the Triton DSL and adds reference code-similarity to correctness and performance. *BackendBench* (Saroufim et al., 2025) evaluates correctness alone for kernels written against the PyTorch backend interface. *MultiKernelBench* (Wen et al., 2025) broadens the platform set to CUDA, Ascend C, and Pallas while retaining the ML-operator workload set.

```

// Dispatch in fft3d_x kernel
threadgroup float2 buf0[128];
threadgroup float2 buf1[128];
if (N == 32u)
    fft_line_32_io(...);
else if (N == 64u)
    fft_line_64_io(..., buf0);
else if (N == 128u)
    fft_line_128_io(..., buf0, buf1);
else {
    // held-out N=256 lands here
    fft_line_direct_fallback_io(...);
}

// fft_line_direct_fallback_io: O(N^2) DFT
float2 acc = float2(0.0f, 0.0f);
float theta = -TWO_PI * float(tid) / float(N);
float2 wstep = float2(cos(theta),
    sin(theta));
float2 w = float2(1.0f, 0.0f);
for (uint n = 0u; n < N; ++n) {
    float2 v = in_data[in_base + n*in_stride];
    acc += cmul(v, w);
    w = cmul(w, wstep);
}
out_data[out_base + tid * out_stride] = acc;
    
```

Figure 4. GPT-5.5 FFT3D iter-10 best: hand-coded `fft_line_32/64/128` routines (left dispatch) deliver the $2.95\times$ in-distribution self-speedup; for any N outside $\{32, 64, 128\}$ the kernel falls into a textbook direct $O(N^2)$ DFT (right). At held-out $N=256$ this costs $\sim 32\times$ more arithmetic per output than the seed’s $O(N \log N)$ Stockham FFT, producing the $0.23\times$ held-out slowdown reported in Table 2. The fast paths reuse a 64-entry constant `float2 W128[]` twiddle table whose stride indexing only covers $N \leq 128$ — the structural reason the fallback is direct DFT rather than a longer FFT.

```

// hmc_step kernel dispatch (GPT-5.5 iter-3 best)
load_A_transpose_tg(A, AT, d, tid, tpg);
threadgroup_barrier(mem_flags::mem_threadgroup);
if (d == 8u) run_hmc_fixed_chunk8<8u>(...);
else if (d == 16u) run_hmc_d16(...); // specialised d=16 path
else if (d == 24u) run_hmc_fixed_chunk8<24u>(...); // covers held-out d
else if (d == 32u) run_hmc_fixed_chunk8<32u>(...);
else
    run_hmc_dynamic(..., d, ...); // runtime-d safety net
    
```

Figure 5. GPT-5.5 HMC iter-3 best: explicit $D \in \{8, 16, 24, 32\}$ enumeration with a runtime- d catch-all. The `run_hmc_fixed_chunk8<24u>` branch is the held-out dimension’s own fully-templated instance — the inner matvec, leapfrog ILP, and per-thread `q[D], p[D]` register allocations all use $D=24$ rather than rounding up to 32 (Opus) or staying runtime (Gemini). In-distribution gmean comes in lower than Opus or Gemini (0.0634 vs 0.0932, 0.0870) — the cost of emitting four template instantiations plus a runtime fallback — but the held-out $d=24$ runs at 10.2% of FP32 peak, $18.6\times$ the seed.

NPUEval (Kalade & Schelle, 2025) targets the AMD AIE NPU in C++ and exposes a tool-use feedback loop with multi-turn regeneration. Most recently, *KernelCraft* (Nie et al., 2026) benchmarks bare-metal assembly kernel generation on emerging accelerator ISAs (PLENA, AMD NPU, Coral NPU, Sonic BOOM), with native tool interfaces (`check_syntax`, `run_evaluation`, `view_output`, `grep_docs`) and a three-tier ML-task taxonomy (primitive, composite, end-to-end). Across these benchmarks the workload is ML and the headline number is speedup against a vendor or compiler baseline.

D.2. LLM-driven evolutionary code search

The broader paradigm of LLMs as *optimisers* inside evolutionary code-search loops was established by *FunSearch* (Romera-Paredes et al., 2023) for symbolic-algorithm discovery and scaled by *AlphaEvolve* (Novikov et al., 2025) across mathematical and algorithmic domains. *AI CUDA Engineer* (Lange et al., 2025) and *EvoEngineer* (Guo et al., 2025) specialise this paradigm to CUDA kernel optimisation, replacing single-shot or agentic regeneration with a population/archive driven by a fitness signal — closer in spirit to our ($\mu=1+\lambda=1$) loop, but on CUDA ML kernels rather than Apple Silicon scientific kernels.

D.3. Where METAL-SCI differs

Existing kernel-generation benchmarks share three structural choices that METAL-SCI deviates from. (i) *Workload domain*. All listed benchmarks target ML operators, where the optimisation surface is dominated by tiling and warp-level reduction patterns heavily represented in pretraining data. METAL-SCI targets scientific compute — regular and irregular stencils, all-pairs interactions, multi-field Boltzmann updates, neighbour-list molecular dynamics, parallel-chain MCMC, multi-kernel nonlinear PDE solvers, and butterfly spectral transforms — six structurally distinct optimisation regimes (R1–R6 of Section 2) whose canonical recipes (Datta et al., 2008; Nyland et al., 2007; Schönherr et al., 2011; Anderson et al., 2008; Govindaraju et al., 2008) do not transfer between regimes. (ii) *Score basis*. All listed benchmarks normalise *achieved* against a vendor or compiler baseline (PyTorch eager, compiler-emitted code, expert hand-tuned reference). This couples the headline number to the strength of that baseline. METAL-SCI normalises against the per-chip roofline ceiling

Table 3. METAL-SCI versus existing LLM kernel-generation benchmarks across the axes that drive the design of our harness. *Domain*: ML = neural-network operators (GEMM, attention, convolution, normalisation, activation); HPC = scientific compute (stencil, N -body, LBM, MD, MCMC, PDE solver, FFT). *Score basis*: what *achieved* is normalised against. *Multi-size*: whether the score requires generalisation across problem sizes. *Search*: the loop structure exposed to the LLM. Within the ML row, all listed benchmarks span a single optimisation regime (matmul-style, with norm/activation epilogues); METAL-SCI spans six structurally distinct regimes (R1–R6 of Section 2).

Benchmark	Target	Domain	Score basis	Multi-size	Search
KernelBench (Ouyang et al., 2025)	CUDA	ML	PyTorch speedup	—	single-shot
TritonBench (Li et al., 2025)	Triton DSL	ML	speedup + code-sim	—	single-shot
BackendBench (Saroufim et al., 2025)	PyTorch backend	ML	correctness only	—	single-shot
MultiKernelBench (Wen et al., 2025)	CUDA / Ascend C / Pallas	ML	compiler speedup	—	multi-turn
NPUeval (Kalade & Schelle, 2025)	AIE C++ (AMD NPU)	ML	compiler speedup	—	tool-use
KernelCraft (Nie et al., 2026)	accelerator asm	ML	compiler speedup	cfg. vars	agentic tool-use
METAL-SCI (ours)	Apple Metal MSL	HPC	% of roofline	3 in + 1 held-out	evolutionary (1+1)

(peak FP32 GFLOPS, peak DRAM GB/s), a hardware-anchored target independent of any reference implementation, with a hard correctness gate ($S=0$ on tolerance failure) and a geometric mean across three problem sizes that discourages overfit to a single working-set regime. (iii) *Hardware*. No listed benchmark targets Apple Silicon’s Metal compute pipeline. Metal is structurally underrepresented in CUDA-centric pretraining data and surfaces a real out-of-distribution generalisation test on Metal-grammar and Metal-stdlib idioms — the `[[max_total_threads_per_threadgroup]]` attribute placement (Sec. 4, seven independent reproductions across five tasks within the opus-4-7 sweep, recurring in earlier opus-4-6 and gemini-3.1-pro runs), `half` as a reserved fp16 keyword, the absence of C++ lambdas, the availability of `simd_shuffle_xor` for intra-simdgroup butterflies, and the absence of `sincospi`. Table 3 summarises these axes against the comparator set.

E. Further results: convergence and iteration budgets

The per-task iteration budget is uneven — lbm 25, wave3d 15, all others 10. Figure 6 shows the running self-speedup *best/seed* per iteration across the three sweeps and justifies the choice. Two patterns recur. First, on most tasks (saxpy, heat2d, nbody, gradshaf, hmc, ising, lj) the incumbent stops moving by iteration ~ 8 for all three models; extending past 10 would not change the headline number. Second, lbm and wave3d are different: lbm’s Opus run plateaus at $1.36\times$ from iter 3 through iter 22, then breaks through to $1.46\times$ at iter 23 (the BGK fold + pinned threadgroup of App. C); a 10-iter budget would have reported $1.36\times$ and missed App. C’s exemplar. GPT-5.5 reaches its lbm best at iter 13 ($1.33\times$). Wave3d’s Opus run shows the same shape with the lift at iter 14 (GPT-5.5 and Gemini stay at $1.00\times$ throughout). One additional late-arriving win shows up only in the new sweep: GPT-5.5’s fft3d best lands at iter 10 ($2.95\times$), having climbed monotonically from iter 2 onward — the only task where extending the GPT-5.5 budget past 10 might have paid off in-distribution (though as the held-out gate shows, that is exactly the win that fails to generalise). Both late-arriving Opus wins land in tasks where the seed is already saturated on at least one size — the search has to pick a structural lever (kernel attribute, marching axis), and 1 + 1 exploration finds those rarely. The figure also makes the silent-correctness story visible: every $\times$ on the $y=1$ baseline is a candidate that compiled or ran wrong, and Opus emits more of them than Gemini or GPT-5.5 at every task with non-trivial search (nbody, hmc, ising, lj, wave3d).

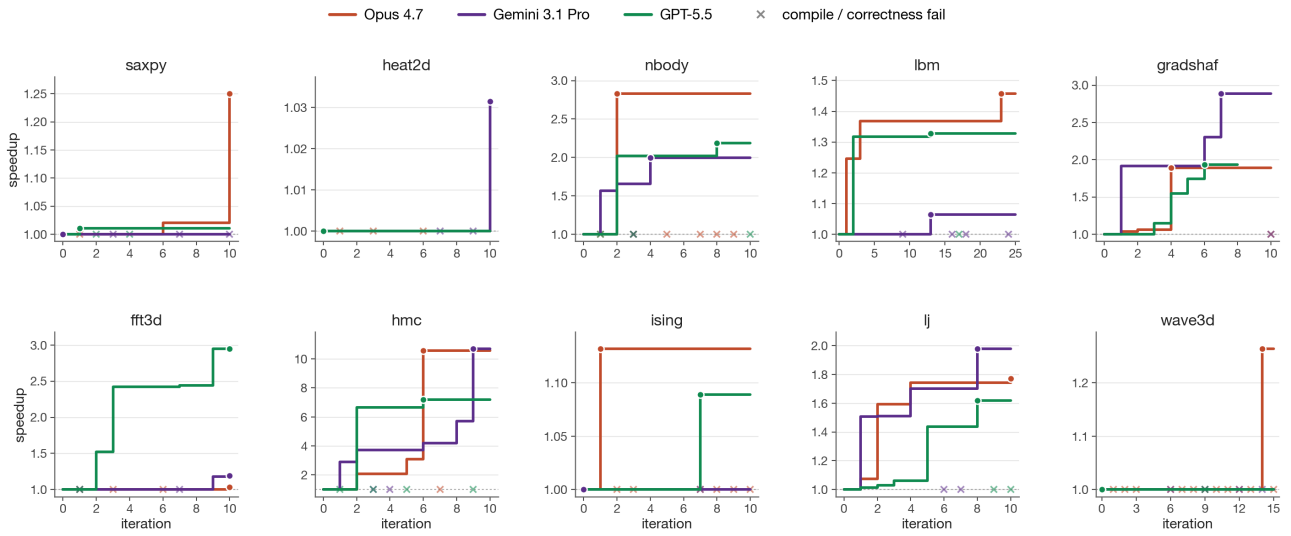


Figure 6. Per-task running self-speedup (best-so-far / seed) versus iteration, Opus 4.7 (purple) vs Gemini 3.1 Pro (green) vs GPT-5.5 (orange). Filled circles mark the iteration that achieved the final best; $\times$ along $y=1$ marks compile or correctness failures (which leave the incumbent unchanged). lbm and wave3d are the tasks where extending the budget past 10 paid off for Opus (lbm iter 23 at $1.46\times$, wave3d iter 14 at $1.26\times$); fft3d is the task where GPT-5.5 keeps climbing through iter 10 (to $2.95\times$). Every other task plateaus by iter ~ 8 for all three models.